HKU Summer Course

Large Language Models

Jintao Ke

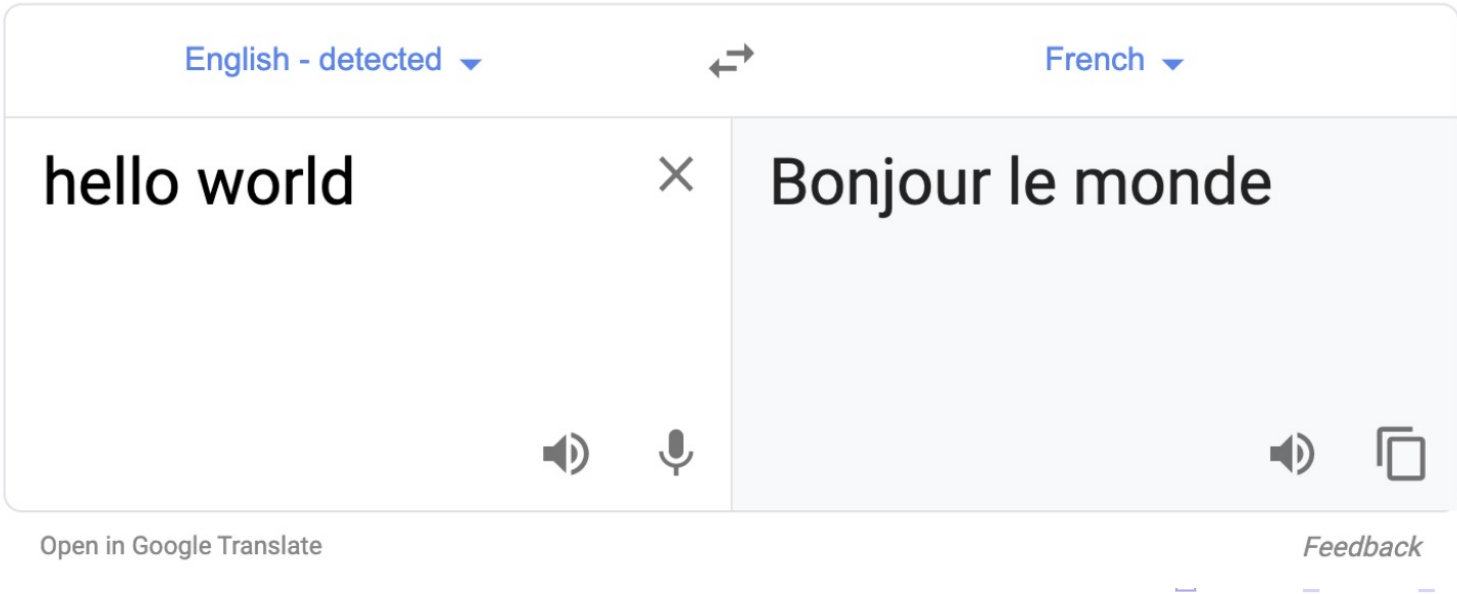
Department of Civil Engineering
Faculty of Engineering
The University of Hong Kong, Pokfulam, Hong Kong
Email: kejintao@hku.hk

Outline

1. Revision of Natural Language Processing (NLP)
   - Concepts of NLP
   - Word Embeddings
2. A Brief Introduction to Transformers
   - Intro to Transformers
   - Attention Mechanism
   - The Transformers Architecture
3. ChatGPT and Large Language Models (LLMs)
   - GPT Models and the ChatGPT Product
   - Training Techniques of GPT
   - Large Language Models (LLMs)
   - Applications of LLMs
   - Prompting, MCPs, and Agents

What Is Natural Language Processing?

- **Natural Language Processing (NLP)** studies how computers process human language.
- Typical tasks include translation, search, spelling correction, sentiment analysis, summarization, and chatbots.
- The key difficulty is that natural language is ambiguous and context-dependent.
- Modern NLP learns patterns from large text corpora instead of relying only on hand-written rules.



NLP Pipeline: From Text to Numbers

1. **Raw text:** a sentence, document, query, or conversation.
2. **Tokenization:** split text into smaller units called tokens.
3. **Embedding:** convert each token into a numerical vector.
4. **Sequence model:** RNNs, LSTMs, Transformers, and LLMs process the vector sequence.
5. **Output:** a label, a score, or another generated sequence.

Key idea

NLP models cannot directly read words. They first turn language into vectors, then learn patterns from those vectors.

Language Model: Predict the Next Token

- Given a token sequence $S = s_1, s_2, \ldots, s_n$, a language model assigns a probability to the sequence:

$$P(S) = P(s_1)P(s_2|s_1)\cdots P(s_n|s_1, \ldots, s_{n-1})$$

- In practice, it repeatedly answers one question:

Next-token prediction

Given the previous tokens $s_1, \ldots, s_{j-1}$, what is the probability that the next token is s_j?

- This same idea supports autocomplete, machine translation, and text generation.

A Simple Example: Word Filling

Fill in the sentence

I painted the car _____.

Possible completions

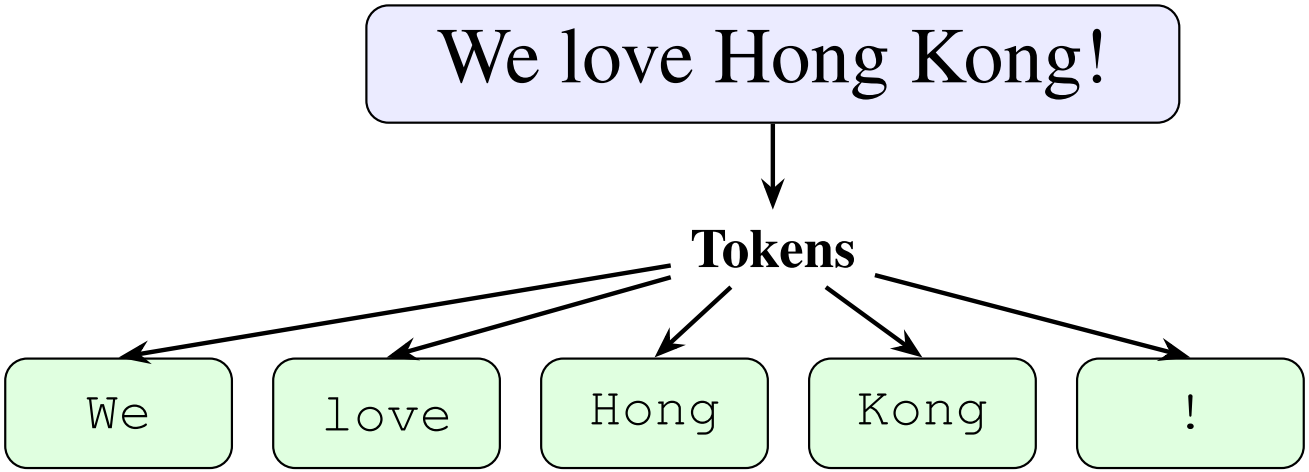
- I painted the car red.
- I painted the car yellow.
- I painted the car gray.

What the model must learn

- "painted" and "car" make color words likely.
- Similar words should have similar numerical representations.
- Context changes which word is most likely.

Tokenization: Split Text into Tokens

- A **token** is a unit that a model reads: a word, punctuation mark, character, or subword piece.
- Tokenization creates a standardized input sequence for NLP models.
- Classical tokenization often splits by spaces and punctuation; modern LLMs often use **subword tokenization**.



Tokenization: Words Are Not Always Tokens

- Example sentence: *AI improves urban traffic control.*
- A word-level tokenizer may produce:

$$[\text{AI},\ \text{improves},\ \text{urban},\ \text{traffic},\ \text{control},\ .]$$

- A subword tokenizer may split uncommon words into pieces, for example:

$$\text{unbelievable} \rightarrow [\text{un},\ \text{believ},\ \text{able}]$$

Why subwords?

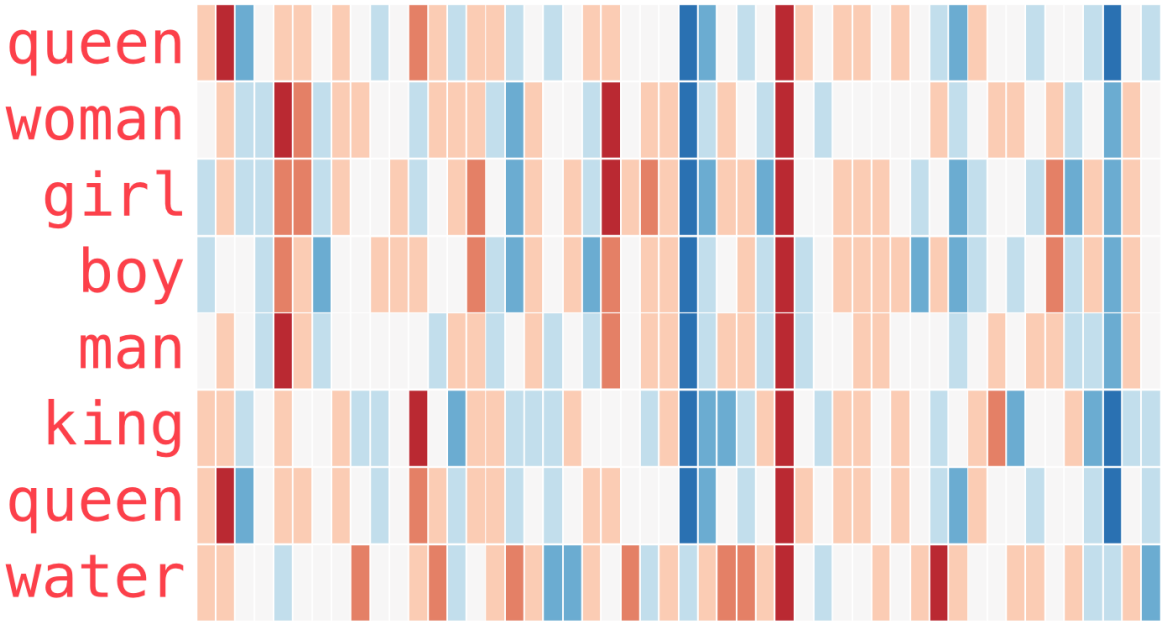
Subwords reduce vocabulary size and help the model handle rare or new words.

Outline

1. Revision of Natural Language Processing (NLP)
   - Concepts of NLP
   - Word Embeddings
2. A Brief Introduction to Transformers
   - Intro to Transformers
   - Attention Mechanism
   - The Transformers Architecture
3. ChatGPT and Large Language Models (LLMs)
   - GPT Models and the ChatGPT Product
   - Training Techniques of GPT
   - Large Language Models (LLMs)
   - Applications of LLMs
   - Prompting, MCPs, and Agents

Word Embedding: Turn Tokens into Vectors

- A word embedding maps each token to a vector: token $\rightarrow \mathbf{e} \in \mathbb{R}^d$.
- In modern neural language models, this is usually a **dense real-valued vector**, not only a 0/1 vector.
- This vector becomes the numerical input to an RNN, LSTM, Transformer, or LLM.



Source: Jay Alammar, The Illustrated Word2vec

Sparse / One-Hot Vectors vs Dense Embeddings

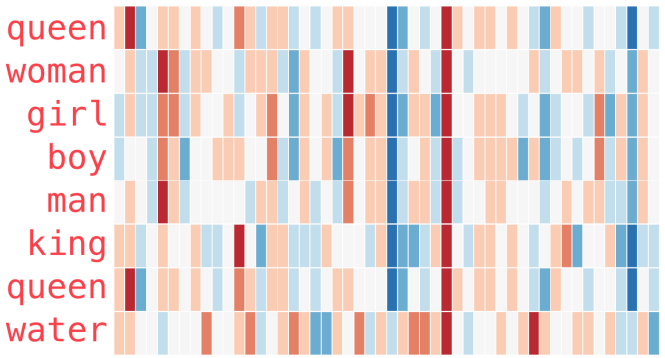
Sparse or one-hot representation

- A one-hot vector has length $|V|$ and only one entry is 1.
- Similarity can still be computed mathematically.
- With dot product or cosine similarity, different words usually have similarity 0, so it cannot express that some different words are semantically closer than others.

man = (0, 1, …, 0, 1, 1, 0, …, 0, 0)
woman = (0, 0, …, 1, …1, 1, 0, 1, …0)
human = (1, 0, …, 1, …1, 1, 0, 0, …1)
manly = (0, 0, …, 1, 1, 0, 0, 1, 1,.. 0)

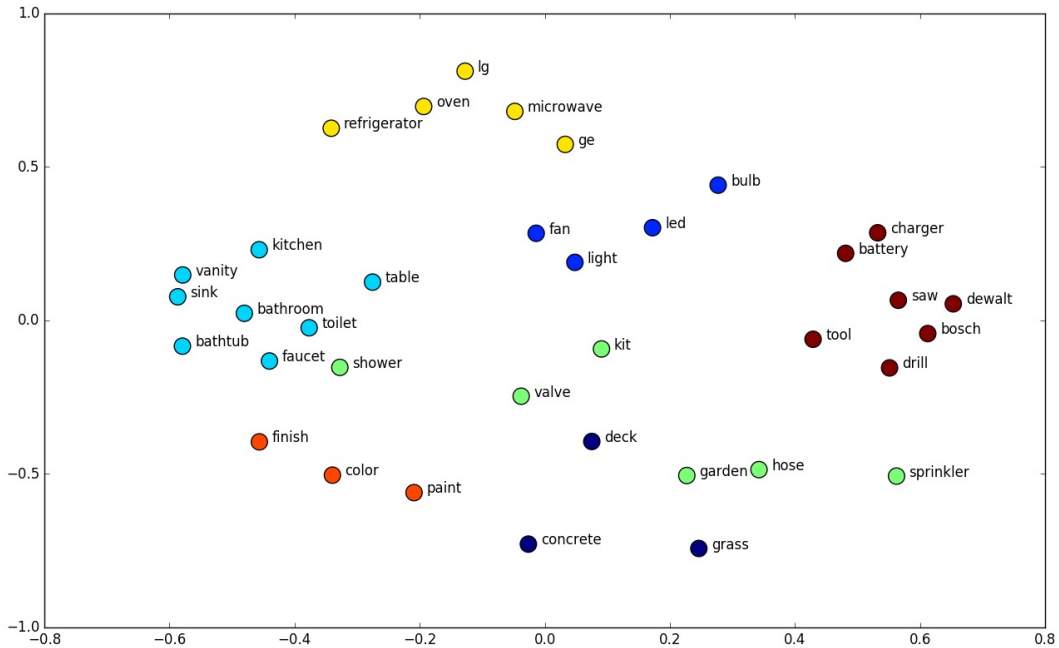
Dense embedding

- Length is a smaller model dimension d.
- Values are real numbers such as 0.23, −1.10, and 0.05.
- Similar meanings are learned to have closer vectors in the embedding space.



What Should Embeddings Learn?

- Words used in similar contexts should be close to each other.
- Related concepts may form clusters: colors, locations, tools, traffic terms, and so on.
- The goal is to make geometry meaningful: distance and direction in vector space should carry language meaning.



How Long Is an Embedding Vector?

- Traditional word embeddings often use dimensions such as $d = 100$, 200, or 300.
- Modern large language models often use much larger token/word embedding dimensions, so each token can carry richer semantic information.
- Published examples:

Model	Dimension	Interpretation
BERT-base	768	hundreds of numbers per token
BERT-large	1024	about one thousand numbers per token
LLaMA-7B	4096	several thousand numbers per token
GPT-3 175B	12288	more than ten thousand numbers per token

Takeaway

In modern RNNs, Transformers, and LLMs, a token is usually represented by a dense vector of many real-valued numbers.

From a Sentence to a Matrix

- Suppose the sentence has five tokens:

$$[\text{AI},\ \text{improves},\ \text{urban},\ \text{traffic},\ \text{control}]$$

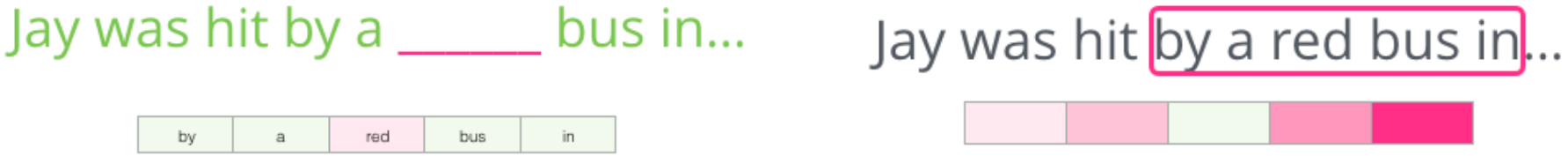
- If each token embedding has dimension d, the input to a Transformer or LLM is:

$$X = \begin{bmatrix} - & - & - & \mathbf{e}_1^T & - & - \\ - & - & - & \mathbf{e}_2^T & - & - \\ - & - & - & \mathbf{e}_3^T & - & - \\ - & - & - & \mathbf{e}_4^T & - & - \\ - & - & - & \mathbf{e}_5^T & - & - \end{bmatrix} \in \mathbb{R}^{5\times d}$$

- A Transformer or LLM may also output a six-token sentence, which can be represented as a $6 \times d$ embedding matrix.

How Are Embeddings Learned?

- **CBOW** predicts the missing target word from its surrounding context words.
- **Skip-gram** does the opposite: it uses a target word to predict nearby context words.
- **GloVe** learns embeddings from global word co-occurrence statistics in a corpus.
- Modern LLMs learn token embeddings together with the whole network while training on large-scale next-token prediction.



Source: Jay Alammar, The Illustrated Word2vec

Embedding Layer: Use a Learned Matrix

- This page explains the **embedding layer inside a Transformer/LLM**, not a separate word2vec training algorithm.
- Tokenization first maps each token to an integer ID.
- The model stores an embedding matrix $E \in \mathbb{R}^{|V| \times d}$; row E_i is the vector for token ID i.
- In modern LLMs, this matrix is trained together with all other model parameters.

Token	Token ID	Embedding vector used by the model
AI	18	$E_{18} = [e_{18,1}, e_{18,2}, \ldots, e_{18,d}]$
traffic	305	$E_{305} = [e_{305,1}, e_{305,2}, \ldots, e_{305,d}]$

Key idea

The embedding layer is a trainable matrix: token ID $\rightarrow$ one row of $E \rightarrow$ vector input to the sequence model.

Outline

1. Revision of Natural Language Processing (NLP)
   - Concepts of NLP
   - Word Embeddings
2. A Brief Introduction to Transformers
   - Intro to Transformers
   - Attention Mechanism
   - The Transformers Architecture
3. ChatGPT and Large Language Models (LLMs)
   - GPT Models and the ChatGPT Product
   - Training Techniques of GPT
   - Large Language Models (LLMs)
   - Applications of LLMs
   - Prompting, MCPs, and Agents

Why Do We Need Transformers?

- RNNs process a sequence step by step: token 1, then token 2, then token 3.
- This makes long-range relationships harder to learn and limits parallel computation.
- Attention improves seq2seq models by letting the decoder look back at all source positions.
- The Transformer takes this idea further: **attention becomes the main operation**, not just an add-on to an RNN.

Core idea

A Transformer represents each token by looking at other relevant tokens in the same context.

Transformer: The Big Picture

1. Text is split into tokens.
2. Each token becomes an embedding vector.
3. Position information is added.
4. Self-attention lets each token collect information from other tokens.
5. Feed-forward layers transform the representation.
6. A final linear layer and softmax produce probabilities for the next token.

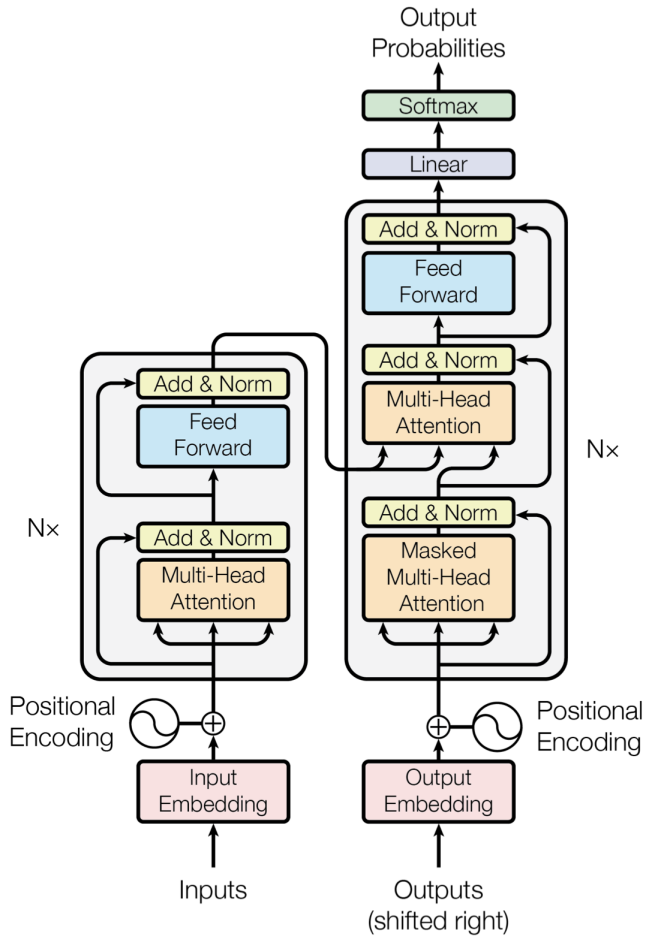


Figure 1: The Transformer - model architecture.

Attention as Information Retrieval

Analogy

- **Query:** what information am I looking for?
- **Key:** what information does each token offer?
- **Value:** the content to copy or combine.

In a sentence

- Each token sends a query.
- It compares this query with all keys.
- Similar keys receive larger attention weights.
- The output is a weighted sum of value vectors.

Intuitive interpretation

Attention answers: *For this token, which other tokens should I use, and how strongly?*

From Attention to Transformer

- **Self-attention:** Q, K, and V all come from the same input sequence.
- **Multi-head attention:** several attention mechanisms run in parallel, so the model can focus on different relationships.
- **Encoder:** builds contextual representations of the input tokens.
- **Decoder:** generates output tokens one by one, using a mask to avoid seeing future tokens.

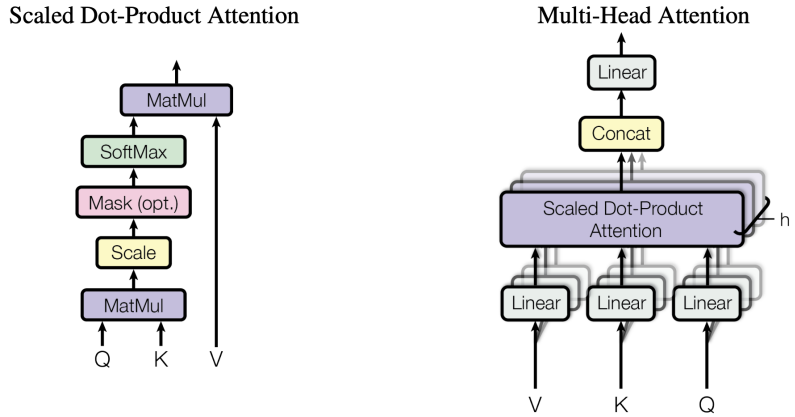


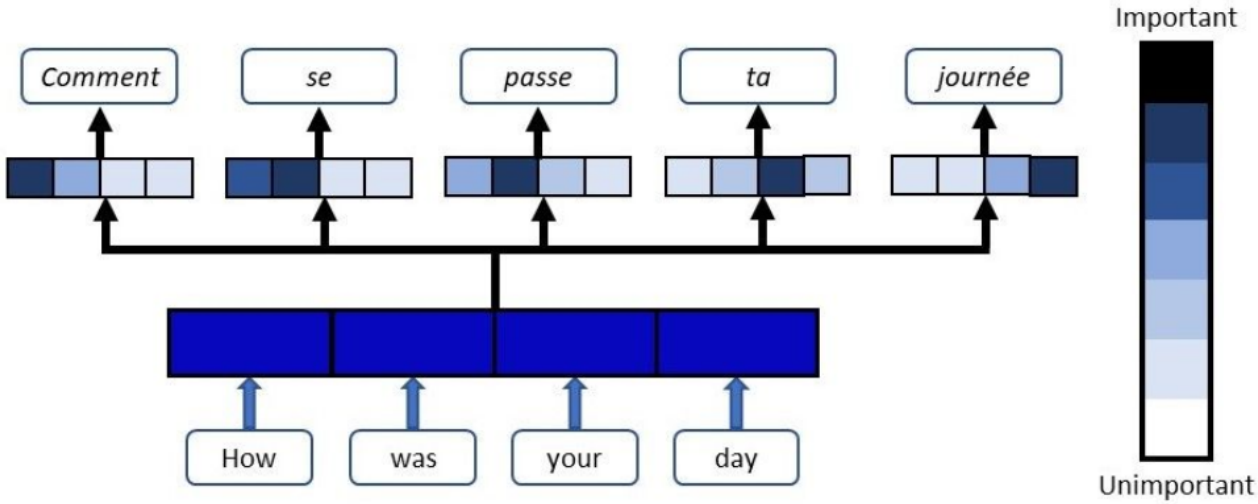
Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

Outline

1. Revision of Natural Language Processing (NLP)
   - Concepts of NLP
   - Word Embeddings
2. A Brief Introduction to Transformers
   - Intro to Transformers
   - Attention Mechanism
   - The Transformers Architecture
3. ChatGPT and Large Language Models (LLMs)
   - GPT Models and the ChatGPT Product
   - Training Techniques of GPT
   - Large Language Models (LLMs)
   - Applications of LLMs
   - Prompting, MCPs, and Agents

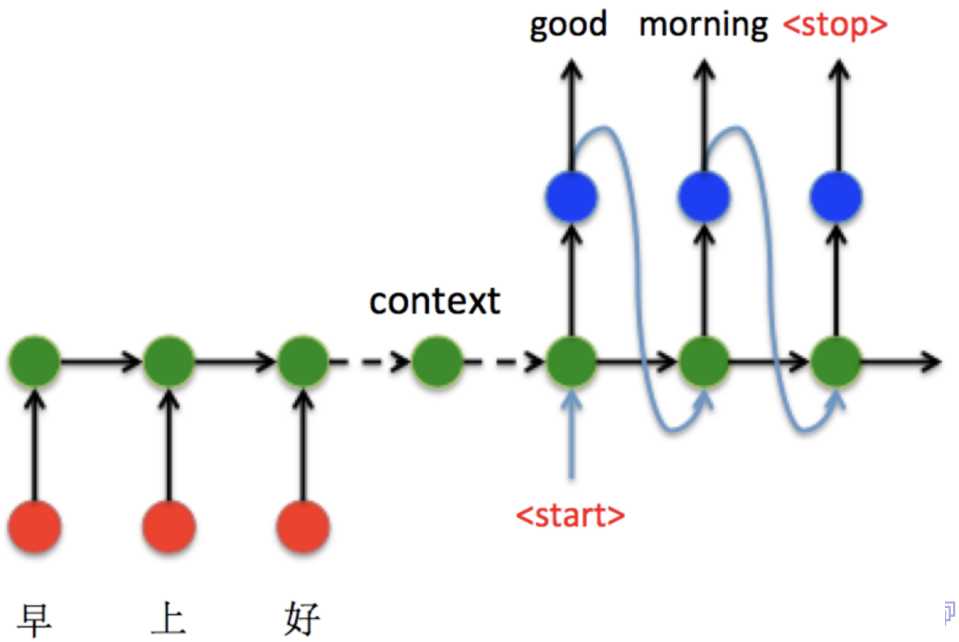
What is Attention in Deep Learning?

- Say we have the sentence “How was your day”, which we would like to translate to the French version - “Comment se passe ta journée”
- What the Attention component of the network will do for each word in the output sentence is map the important and relevant words from the input sentence and assign higher weights to these words, enhancing the accuracy of the output prediction.



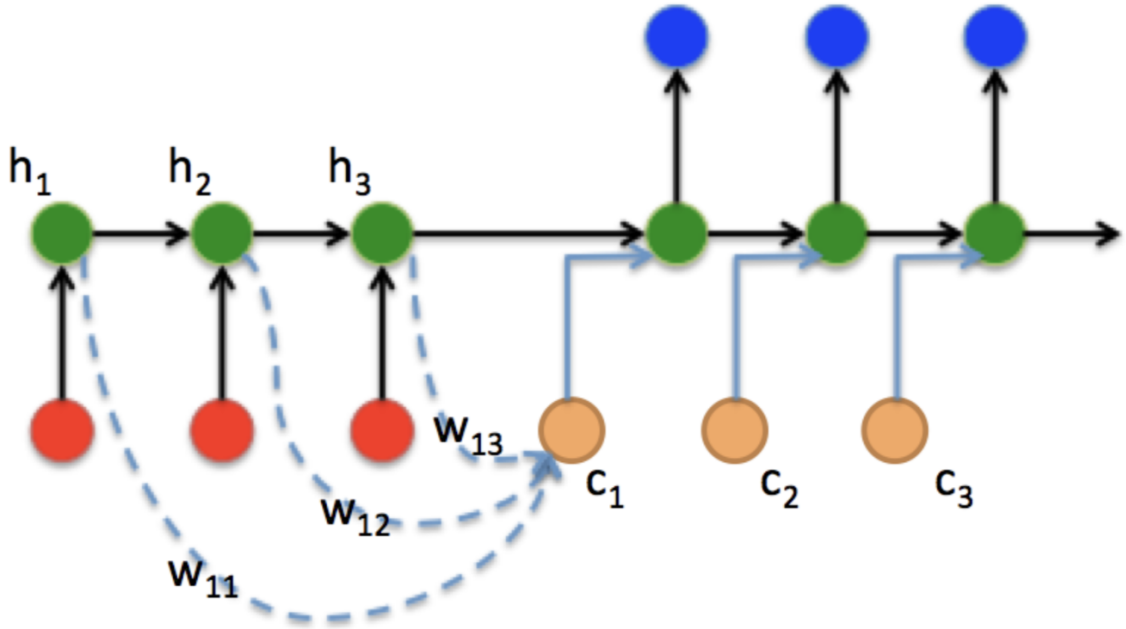
Challenges of Sequence to Sequence Learning

- The standard seq2seq model encodes all input sequences into a single semantic vector *Context* through a series of encoders (usually RNNs), which are then decoded by Decoders.
- *Context* is the only input to the decoder and contains all the information in the original sequence.
  - The length of *Context* limits the performance of the model.
  - The decoder only uses the last hidden layer state of the encoder, which results in low information utilization.



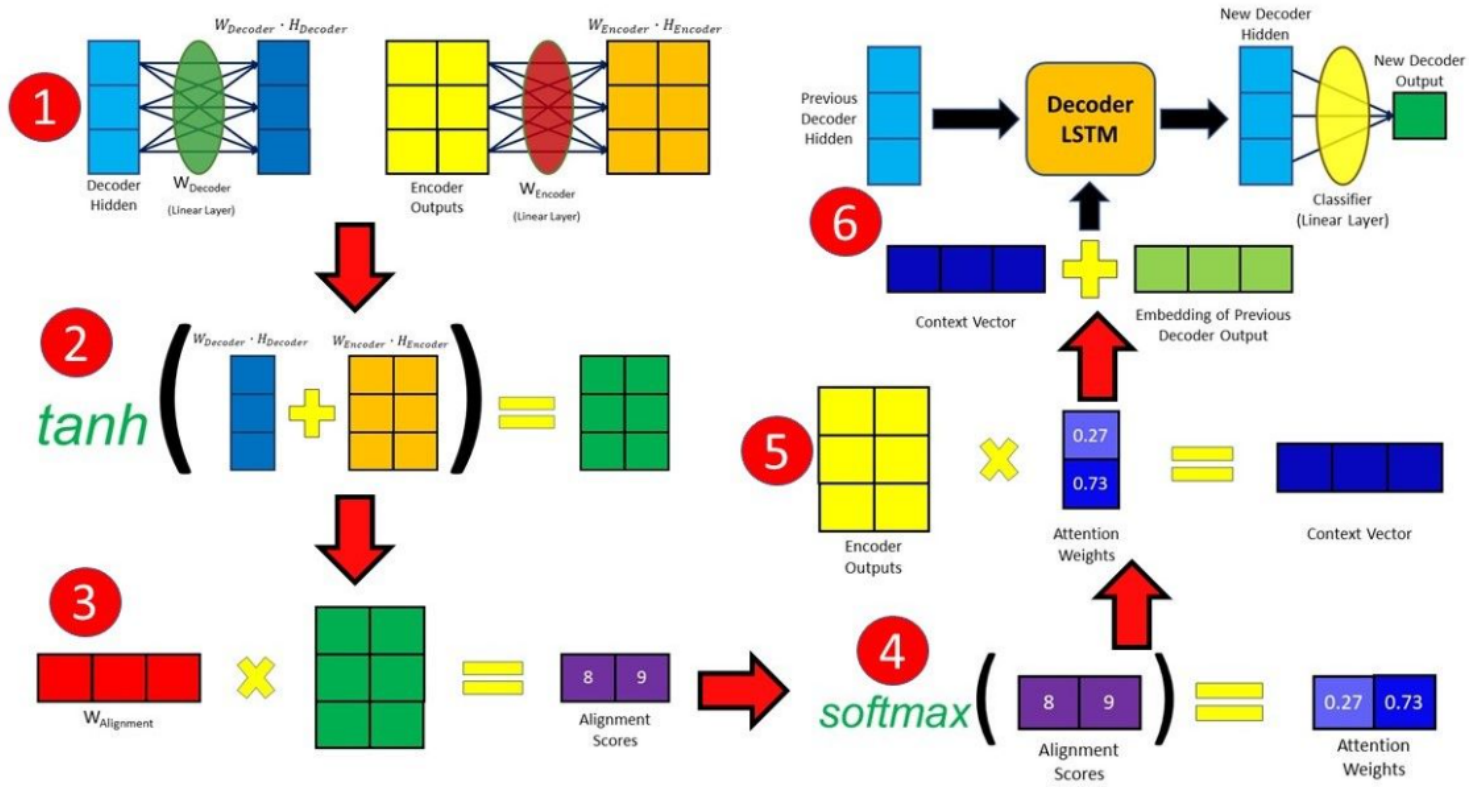
Challenges of Sequence to Sequence Learning

- A direct improvement idea is to use all hidden layer states of the Encoders to solve the *Context* length limitation problem.
- Attention mechanism does it by creating a weighted average of all the encoder hidden states.
- For each decoder, its input comes from the output of the previous decoder and the information of the entire input sequence created by the attention mechanism.



Attention Weights

- The attention weights that assign different levels of focus on the decoded inputs are learned together with the training of the neural networks, instead of preset.



How to Calculate Attention I

Start with a single value example

- Given a key-value table, for a query key q not in the table, we predict by a weighted average of available values.

$$f(q) = \sum_{i=1}^{N} \alpha(q, k_i) v_i \tag{1}$$

- $\alpha(q, k_i)$ measures how relevant key k_i is to query q; softmax gives closer keys larger weights.

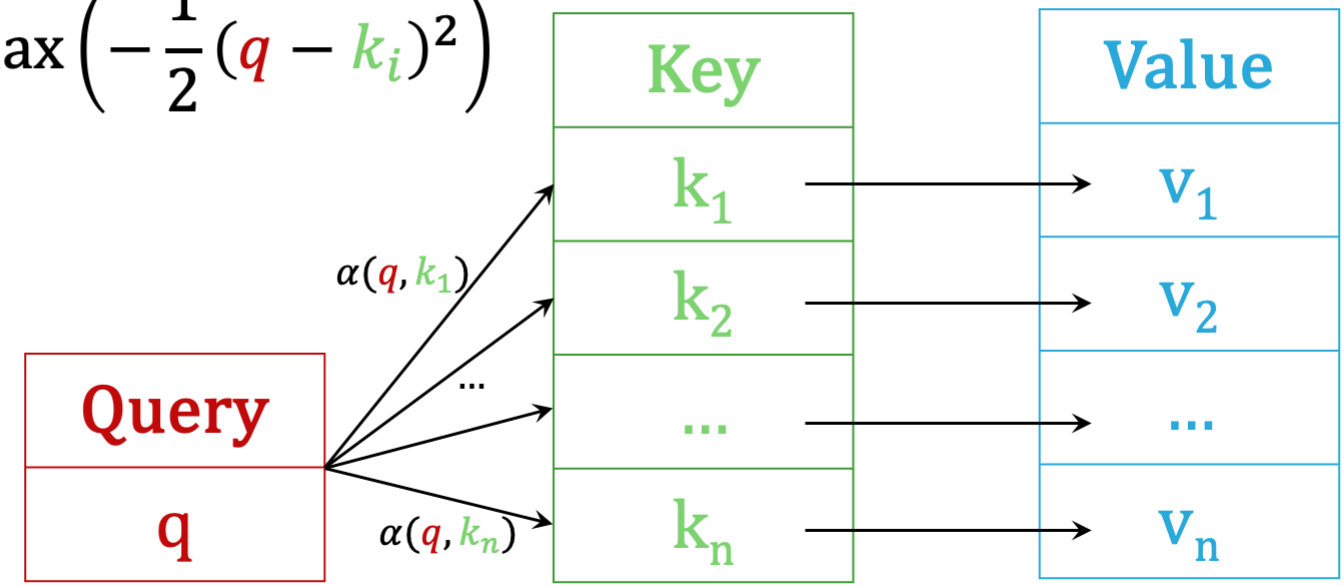
$$\begin{aligned} \alpha(q, k_i) &= \text{softmax}\left(-\frac{1}{2}(q - k_i)^2\right) \\ &= \frac{\exp\left(-\frac{1}{2}(q - k_i)^2\right)}{\sum_{j=1}^{N} \exp\left(-\frac{1}{2}(q - k_j)^2\right)} \end{aligned} \tag{2}$$

How to Calculate Attention II

Start with a single value example

$$f(q) = \alpha(q, k_1)v_1 + \alpha(q, k_2)v_2 + \cdots + \alpha(q, k_n)v_n$$

$$\alpha(q, k_i) = \text{softmax}\left(-\frac{1}{2}(q - k_i)^2\right)$$



How to Calculate Attention III

Expand to vectors

- When each query, key, and value is a (row) vector, the idea is similar.

$$f(\mathbf{q}) = \sum_{i=1}^{N} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v_i} \tag{3}$$

- At this point we can use the dot product of two vectors to represent the similarity between them, which is called **dot-product attention**.
- To alleviate the vanishing gradient problem, the dot product can be divided by the dimension of the key vector, which is the **scaled dot-product attention** coefficient.

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \text{softmax}\left(\mathbf{q}\mathbf{k}_i^\top / \sqrt{d}\right) \tag{4}$$

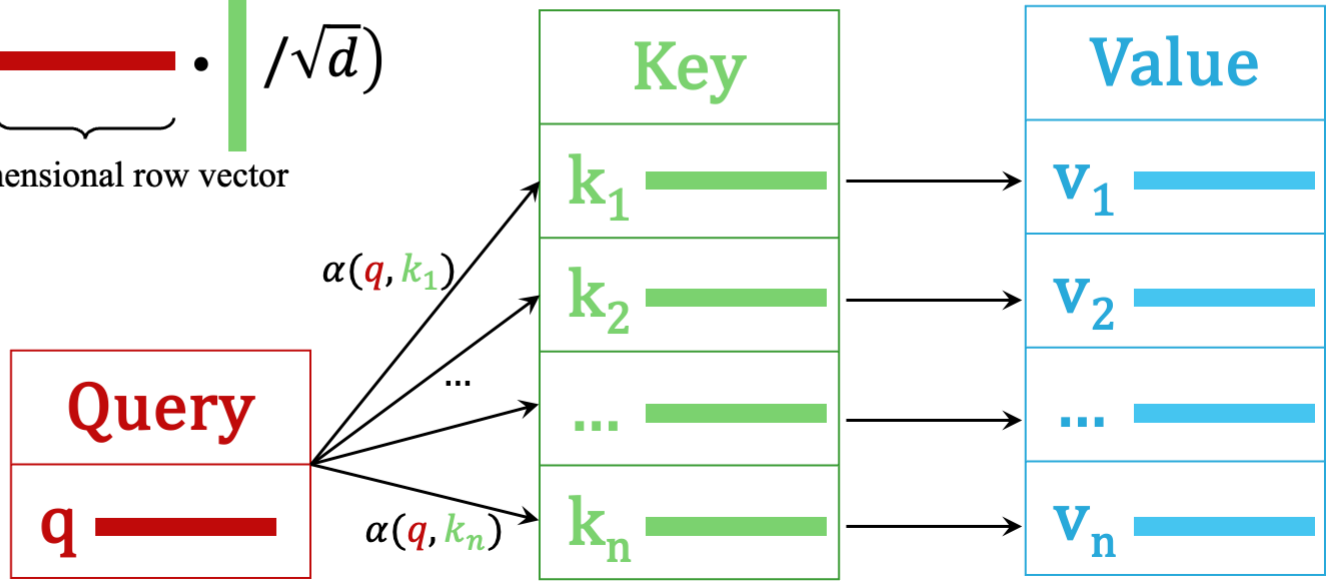
How to Calculate Attention IV

Expand to vectors

$$f(\mathbf{q}) = \alpha(\mathbf{q}, \mathbf{k}_1)\mathbf{v}_1 + \alpha(\mathbf{q}, \mathbf{k}_2)\mathbf{v}_2 + \cdots + \alpha(\mathbf{q}, \mathbf{k}_n)\mathbf{v}_n$$

$$\alpha(q, k_i) = \text{softmax}(\mathbf{q}\mathbf{k}_i^\top/\sqrt{d})$$

$$= \text{softmax}(\underbrace{\text{——}}_{\text{d-dimensional row vector}} \cdot \text{|} /\sqrt{d})$$



How to Calculate Attention V

Multi queries and matrix form

- When there are multiple queries (row vectors $\mathbf{q}_i$), the above process can be quickly completed via matrix multiplication.

$$Q_{(m\times d_k)} = \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \\ \vdots \\ \mathbf{q}_m \end{bmatrix}, K_{(n\times d_k)} = \begin{bmatrix} \mathbf{k}_1 \\ \mathbf{k}_2 \\ \vdots \\ \mathbf{k}_n \end{bmatrix}, V_{(n\times d_v)} = \begin{bmatrix} \mathbf{v}_1 \\ \mathbf{v}_2 \\ \vdots \\ \mathbf{v}_n \end{bmatrix} \tag{5}$$

- m is the number of queries; n is the number of key-value pairs; d_k is the dimension of the key vector, also the dimension of the query vector as the query vector and key vector should have the same dimensions; and d_v dimension of the value vector.

How to Calculate Attention VI

Multi queries and matrix form

$$QK^\top = \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \\ \vdots \\ \mathbf{q}_m \end{bmatrix} \begin{bmatrix} \mathbf{k}_1^\top & \mathbf{k}_2^\top & \cdots & \mathbf{k}_n^\top \end{bmatrix} = \begin{bmatrix} \mathbf{q}_1\mathbf{k}_1^\top & \cdots & \mathbf{q}_1\mathbf{k}_n^\top \\ \vdots & \ddots & \vdots \\ \mathbf{q}_m\mathbf{k}_1^\top & \cdots & \mathbf{q}_m\mathbf{k}_n^\top \end{bmatrix} \tag{6}$$

$$\text{softmax}(QK^\top/\sqrt{d_k}) = \begin{bmatrix} \alpha(\mathbf{q}_1,\mathbf{k}_1) & \cdots & \alpha(\mathbf{q}_1,\mathbf{k}_n) \\ \vdots & \ddots & \vdots \\ \alpha(\mathbf{q}_m,\mathbf{k}_1) & \cdots & \alpha(\mathbf{q}_m,\mathbf{k}_n) \end{bmatrix} \tag{7}$$

(Here, the softmax function means performing the softmax operation row by row.)

How to Calculate Attention VII

Multi queries and matrix form

$$\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V = \begin{bmatrix} \sum_{i=1}^{n} \alpha(\mathbf{q}_1, \mathbf{k}_i)\mathbf{v}_i \\ \vdots \\ \sum_{i=1}^{n} \alpha(\mathbf{q}_m, \mathbf{k}_i)\mathbf{v}_i \end{bmatrix} = \begin{bmatrix} f(\mathbf{q}_1) \\ \vdots \\ f(\mathbf{q}_m) \end{bmatrix}_{(m \times d_v)} \tag{8}$$

Thus we obtain the scaled dot-product attention estimations $f(\mathbf{q}_1), f(\mathbf{q}_2), ... f(\mathbf{q}_m)$ corresponding to the queries $\mathbf{q}_1$, $\mathbf{q}_2$, ..., $\mathbf{q}_m$ in the form of an $m \times d_v$ matrix.

Example of Scaled Dot-Product Attention Calculation

- Example: Given a query vector $\mathbf{q} = (0, 1, 1)$, three key vectors $\mathbf{k}_1 = (1, 0, 1)$,$\mathbf{k}_2 = (0, 1, 0)$,$\mathbf{k}_3 = (1, 0, 0)$ and the corresponding value vector $\mathbf{v}_1 = (1, 2)$, $\mathbf{v}_2 = (0, 3)$, $\mathbf{v}_3 = (1, 0)$. Use the scaled dot-product attention to predict the value for query $\mathbf{q}$.
- Solution:
  1. Scaling factor $\sqrt{d_k} = \sqrt{3}$, calculate the scaled correlation: $\mathbf{q}\mathbf{k}_1^{\mathrm{T}}/\sqrt{d_k} = 0.577, \quad \mathbf{q}\mathbf{k}_2^{\mathrm{T}}/\sqrt{d_k} = 0.577, \quad \mathbf{q}\mathbf{k}_3^{\mathrm{T}}/\sqrt{d_k} = 0$
  2. Normalized Scores:

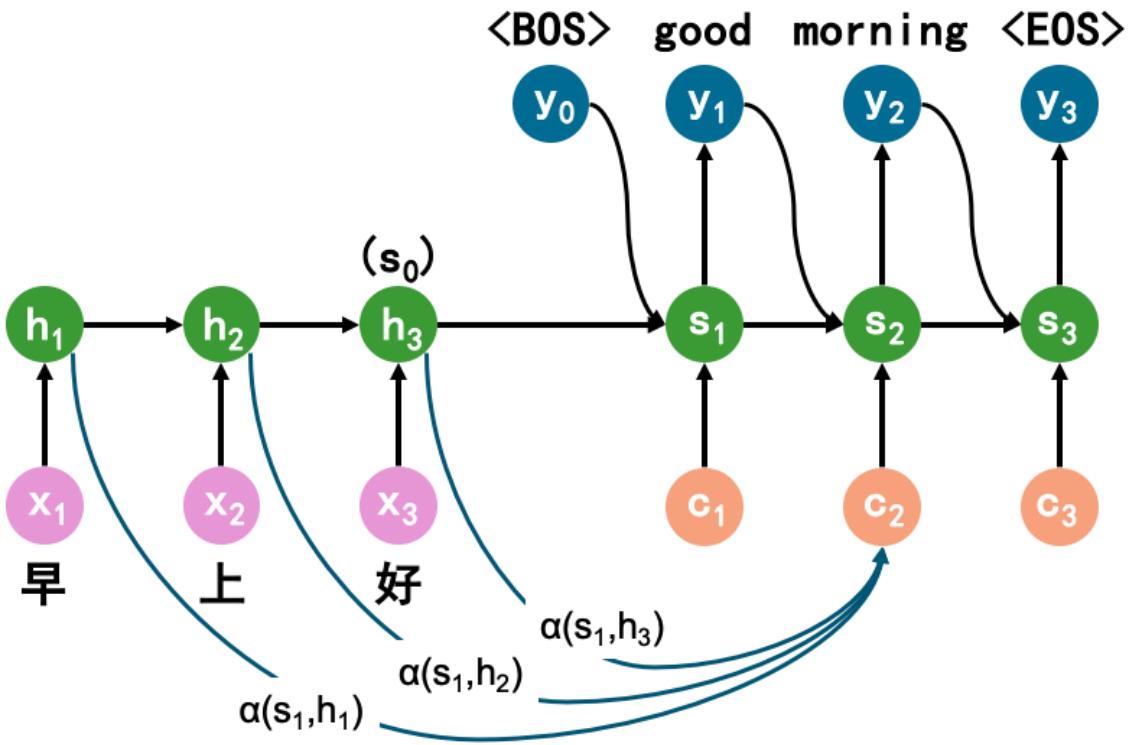
$$\begin{bmatrix} \alpha(\mathbf{q}, \mathbf{k}_1) \\ \alpha(\mathbf{q}, \mathbf{k}_2) \\ \alpha(\mathbf{q}, \mathbf{k}_3) \end{bmatrix} = \text{softmax}\left(\begin{bmatrix} \mathbf{q}\mathbf{k}_1^{\mathrm{T}}/\sqrt{d_k} \\ \mathbf{q}\mathbf{k}_2^{\mathrm{T}}/\sqrt{d_k} \\ \mathbf{q}\mathbf{k}_3^{\mathrm{T}}/\sqrt{d_k} \end{bmatrix} \right) = \begin{bmatrix} 0.39 \\ 0.39 \\ 0.22 \end{bmatrix}$$

  3. Calculate the Prediction: $f(\mathbf{q}) = \sum_{i=1}^{3} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i = (0.61, 1.95)$

Inferring Process of Seq2Seq with Attention

- In a seq2seq model with attention, the decoder hidden state s_t at each time step t depends on the output of the previous time step y_t, the hidden state of the previous time step s_{t-1}, and a unique context vector c_t (instead of the previous unified context vector c).

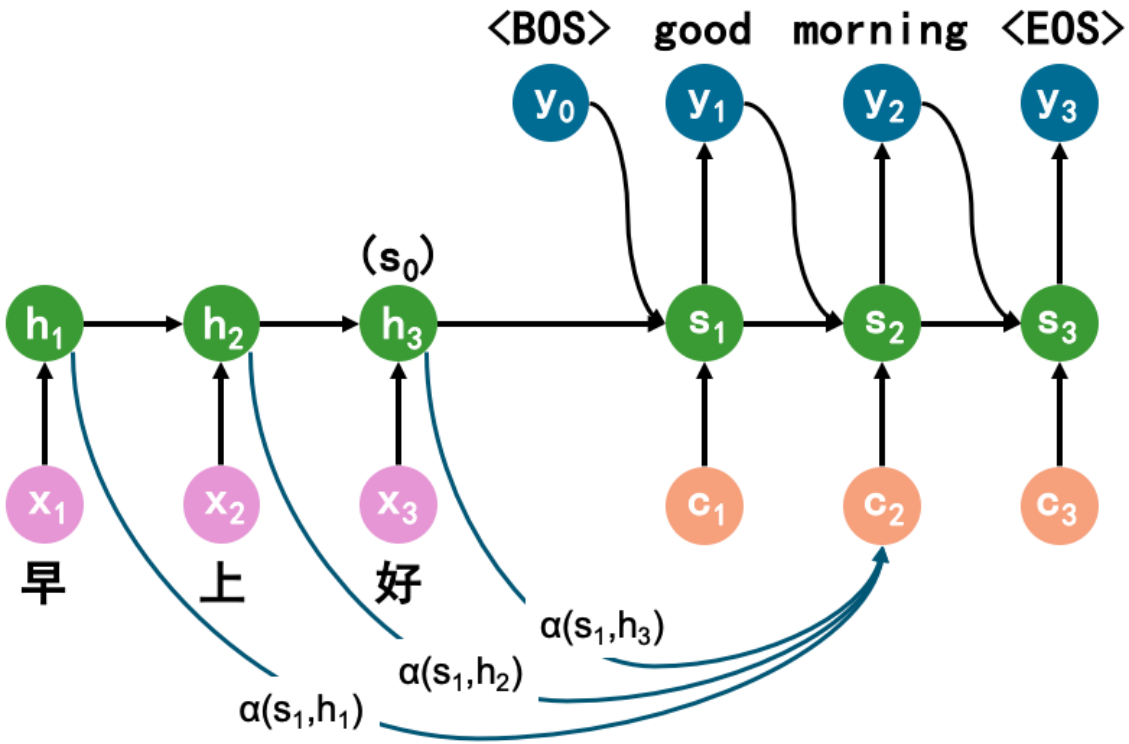
$$s_t = f(s_{t-1}, y_{t-1}, c_t) \tag{9}$$



Inferring Process of Seq2Seq with Attention

- The context vector c_t is calculated through the Attention Mechanism using the hidden state of the previous time step s_{t-1} as the Query and ALL the hidden states of the encoder $\mathbf{h}$ as the Key and Value.

$$c_t = \sum_j \alpha(s_{t-1}, h_j) \cdot h_j \tag{10}$$



Outline

1. Revision of Natural Language Processing (NLP)
   - Concepts of NLP
   - Word Embeddings
2. A Brief Introduction to Transformers
   - Intro to Transformers
   - Attention Mechanism
   - The Transformers Architecture
3. ChatGPT and Large Language Models (LLMs)
   - GPT Models and the ChatGPT Product
   - Training Techniques of GPT
   - Large Language Models (LLMs)
   - Applications of LLMs
   - Prompting, MCPs, and Agents

What Is Input for LLMs?

- After tokenization, a sentence is a sequence of L tokens.
- After embedding, each token is represented by a vector of length d_{model}.

$$[\text{AI},\ \text{helps},\ \text{transportation},\ .] \quad \Longrightarrow \quad X = \begin{bmatrix} - & - & - & \mathbf{x}_1^T & - & - & - \\ - & - & - & \mathbf{x}_2^T & - & - & - \\ & & & \vdots & & & \\ - & - & - & \mathbf{x}_L^T & - & - & - \end{bmatrix} \in \mathbb{R}^{L \times d_{\text{model}}}$$

Interpretation

Each row is one token position; each column is one feature dimension of the token embedding.

Single-Head Self-Attention: Weight Shapes

- Start with $X \in \mathbb{R}^{L \times d_{\text{model}}}$.
- Self-attention learns three projections:

$$W^Q, W^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

$$Q = XW^Q \in \mathbb{R}^{L \times d_k}, \quad K = XW^K \in \mathbb{R}^{L \times d_k}, \quad V = XW^V \in \mathbb{R}^{L \times d_v}$$

Meaning

The model creates query, key, and value vectors for every token. These are not new tokens; they are new feature views of the same L tokens.

Single-Head Self-Attention: Output Shape

- Attention scores compare every token with every token:

$$QK^T: \quad (L \times d_k)(d_k \times L) = L \times L$$

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \in \mathbb{R}^{L \times L}$$

$$\text{Attention}(Q, K, V) = AV: \quad (L \times L)(L \times d_v) = L \times d_v$$

When does the shape match X?

Single-head attention outputs $L \times d_v$. It has the same shape as X only if $d_v = d_{\text{model}}$, or if an output projection maps it back to d_{model}.

Multi-Head Self-Attention: Shape Logic

- With h heads, usually $d_{\text{head}} = d_{\text{model}}/h$.
- For each head i:

$$W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{head}}}$$

$$Q_i, K_i, V_i \in \mathbb{R}^{L \times d_{\text{head}}}, \quad \text{head}_i \in \mathbb{R}^{L \times d_{\text{head}}}$$

$$[\text{head}_1, \ldots, \text{head}_h] \in \mathbb{R}^{L \times (hd_{\text{head}})} = \mathbb{R}^{L \times d_{\text{model}}}$$

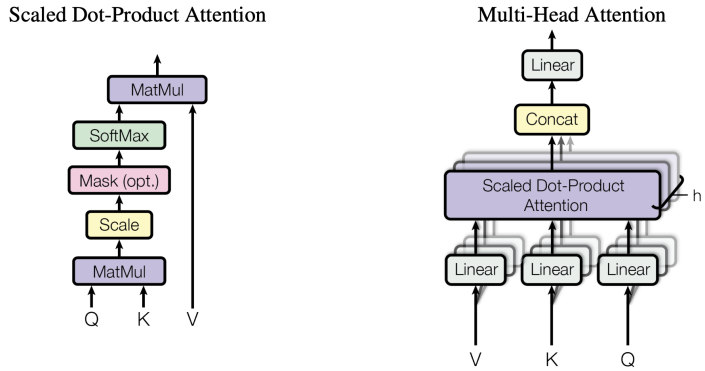


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

Multi-Head Attention: Final Projection

- After concatenating all heads, the model applies one more output projection:

$$W^O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$$

$$(L \times d_{\text{model}})(d_{\text{model}} \times d_{\text{model}}) = L \times d_{\text{model}}$$

Key takeaway

Multi-head attention is designed so that

$$X \in \mathbb{R}^{L \times d_{\text{model}}} \quad \Longrightarrow \quad \text{MultiHead}(X) \in \mathbb{R}^{L \times d_{\text{model}}}.$$

Why Keep the Shape Unchanged?

- **Layer stacking:** the next Transformer layer expects the same $L \times d_{\text{model}}$ input shape.
- **Residual connection:** $X + \text{Sublayer}(X)$ requires matching shapes.
- **Token alignment:** row i still corresponds to token i, but with richer context.
- **Parallel computation:** all token vectors can be updated together as one matrix.

Intuition

Attention changes the **content** of each token vector, not the number of tokens or the model dimension.

Example: Shape Check

- Suppose $L = 10$, $d_{\text{model}} = 512$, $h = 8$, so $d_{\text{head}} = 64$.

$$X \in \mathbb{R}^{10\times 512}, \quad W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{512\times 64}$$

$$Q_i, K_i, V_i \in \mathbb{R}^{10\times 64}, \quad Q_i K_i^T \in \mathbb{R}^{10\times 10}$$

$$\text{head}_i \in \mathbb{R}^{10\times 64}, \quad [8 \text{ heads}] \in \mathbb{R}^{10\times (8\cdot 64)} = \mathbb{R}^{10\times 512}$$

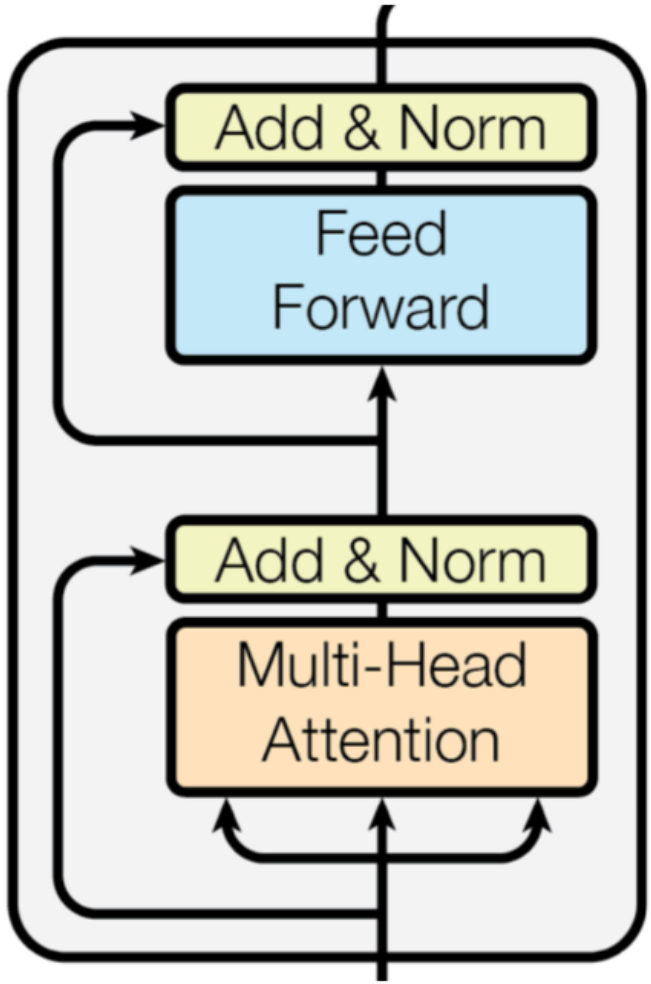
$$(10 \times 512)(512 \times 512) = 10 \times 512$$

Transformer Layer: A Repeated Block

- A layer updates token vectors using:
  - multi-head attention,
  - feed-forward network,
  - residual connection,
  - normalization.
- The feed-forward network usually expands then compresses:

$$d_{\text{model}} \to d_{\text{ff}} \to d_{\text{model}}.$$

- Therefore the whole layer keeps shape $L \times d_{\text{model}}$, so layers can be stacked.



Positional Encoding

- Self-attention sees token vectors, but does not know their order by itself.
- Positional encoding adds position information to token embeddings.

$$\text{input to Transformer} = \text{token embedding} + \text{position encoding}$$

Why it matters

"traffic congestion causes delay" and "delay causes traffic congestion" contain similar words but different order and meaning.

Encoder and Decoder in the Original Transformer

Encoder

- Reads all source tokens.
- Builds contextual representations used by the decoder.

Decoder

- Generates tokens one by one.
- Uses a mask to avoid future tokens.
- Uses cross-attention to read encoder outputs.

Output probabilities

- Linear + softmax converts decoder output to a probability distribution over the vocabulary.
- The next token is selected from this distribution.

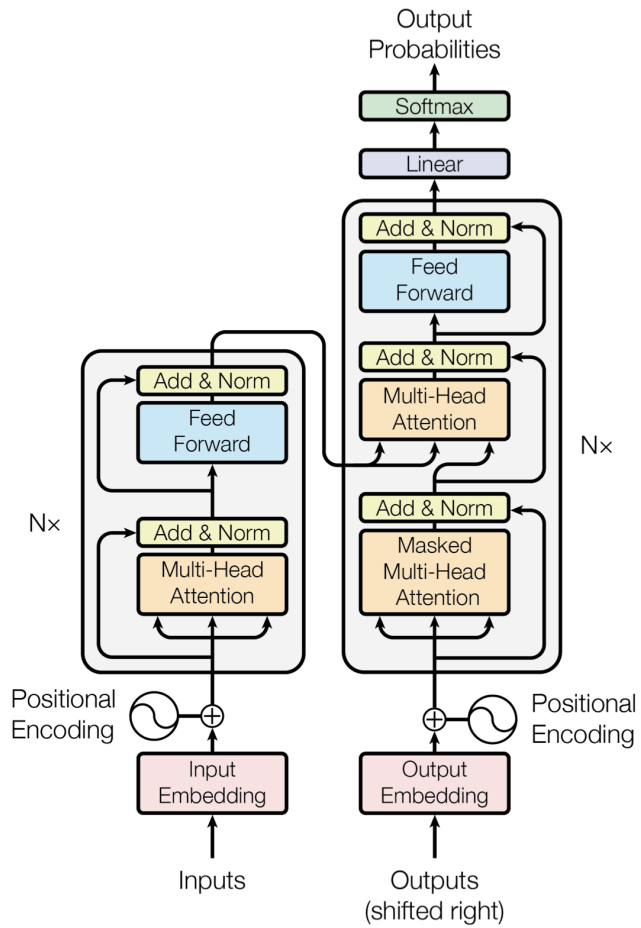


Figure 1: The Transformer - model architecture.

Decoder Mask: Why Future Tokens Are Hidden

- The mask is added to the attention scores, also called logits, before softmax.
- Allowed positions add 0, so their scores do not change.
- Future positions add $-\infty$, so after softmax they receive zero attention.

$$S_{\text{masked}} = \frac{QK^T}{\sqrt{d_k}} + M, \quad M = \begin{bmatrix} 0 & -\infty & -\infty & \cdots \\ 0 & 0 & -\infty & \cdots \\ 0 & 0 & 0 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

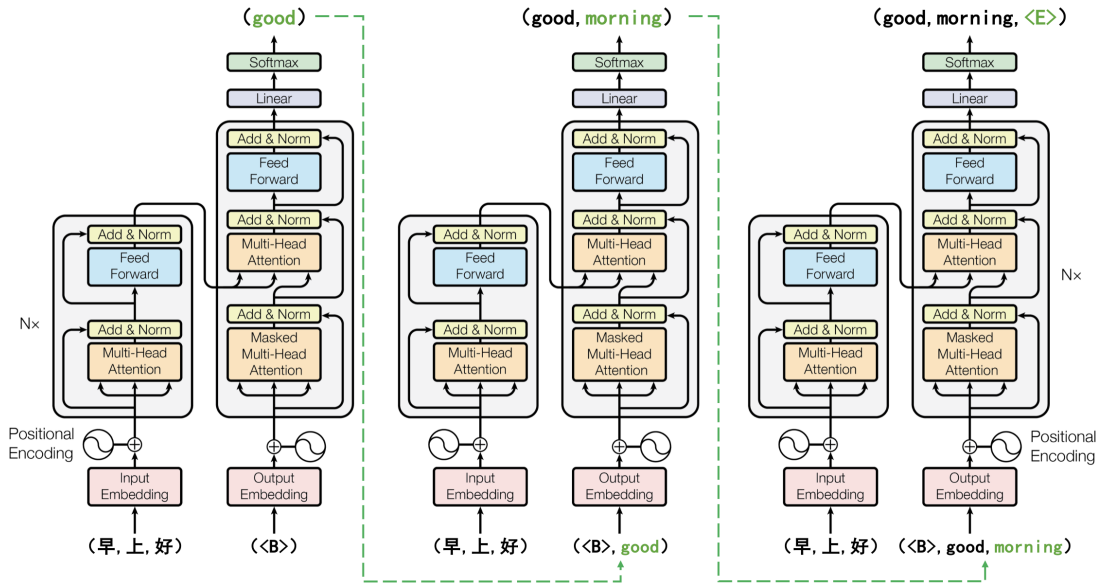
$$\text{softmax}(S_{\text{masked}}) \quad \text{where} \quad \exp(-\infty) = 0$$

Key idea

Tokens cannot attend to future tokens because their softmax weights become zero. This makes training consistent with left-to-right generation.

How Data Flows in a Transformer

1. Text becomes token embeddings plus positions.
2. Transformer layers update token vectors.
3. A linear layer and softmax predict the next token.
4. The selected token is appended; repeat.



Takeaways on the Transformer Architecture

- In NLP/LLMs, X is a token embedding matrix: $L \times d_{\text{model}}$.
- Self-attention first produces scores of shape $L \times L$: every token compares with every token.
- Multi-head attention preserves the final shape through concatenation and W^O.
- Keeping the shape unchanged enables residual connections and deep stacking.
- Masked decoder self-attention enables next-token generation.

Common Transformer Structures

Structure	Main use	Examples
Encoder-only	Understand an input sequence	BERT-style models
Encoder-decoder	Map one sequence to another	original Transformer, T5-style models
Decoder-only	Generate the next token	GPT, LLaMA, many modern LLMs

Modern LLMs

Most mainstream chat/generative LLMs use a **decoder-only Transformer** because next-token prediction naturally matches text generation.

Pre-training and Fine-tuning

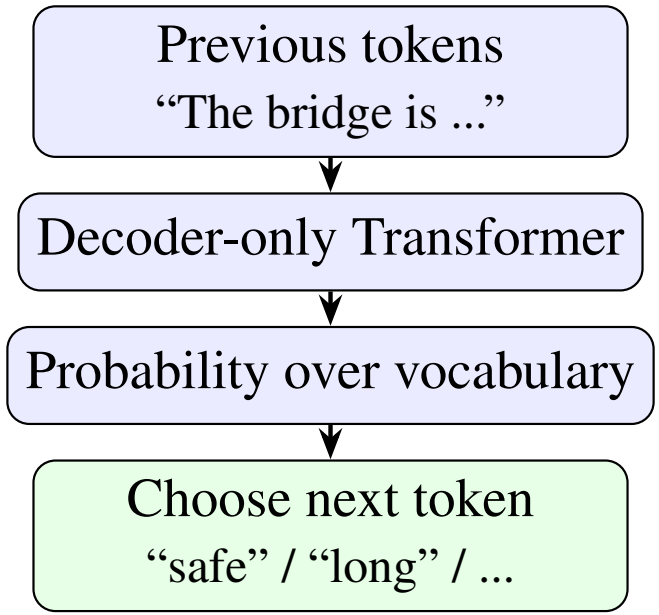
- Training a large language model from scratch needs massive text data and computation.
- **Pre-training**: learn general language patterns by predicting tokens on large corpora.
- **Fine-tuning**: adapt the pretrained model to a task, domain, or instruction-following style.
- This two-stage workflow is more efficient than training a specialized model from scratch.

Outline

1. Revision of Natural Language Processing (NLP)
   - Concepts of NLP
   - Word Embeddings
2. A Brief Introduction to Transformers
   - Intro to Transformers
   - Attention Mechanism
   - The Transformers Architecture
3. ChatGPT and Large Language Models (LLMs)
   - GPT Models and the ChatGPT Product
   - Training Techniques of GPT
   - Large Language Models (LLMs)
   - Applications of LLMs
   - Prompting, MCPs, and Agents

What Does GPT Mean?

- GPT = **Generative Pre-trained Transformer**.
- **Generative**: produces new text token by token.
- **Pre-trained**: first learns from massive text before being adapted to tasks.
- **Transformer**: uses the decoder part of the Transformer architecture.



Core idea

A GPT model repeatedly answers one simple question: what token is likely to come next?

Why GPT Was the Starting Point of LLMs

- Before GPT, many NLP systems were trained for **one specific task** only.
- GPT showed a different route: train one large general model first, then adapt it to many tasks.
- GPT-3 showed that a single model could write, summarize, translate, code, and answer many types of questions.
- ChatGPT made this capability widely recognized by the public through a simple conversational product.
- This is why GPT is often viewed as the start point of modern LLMs by many people.

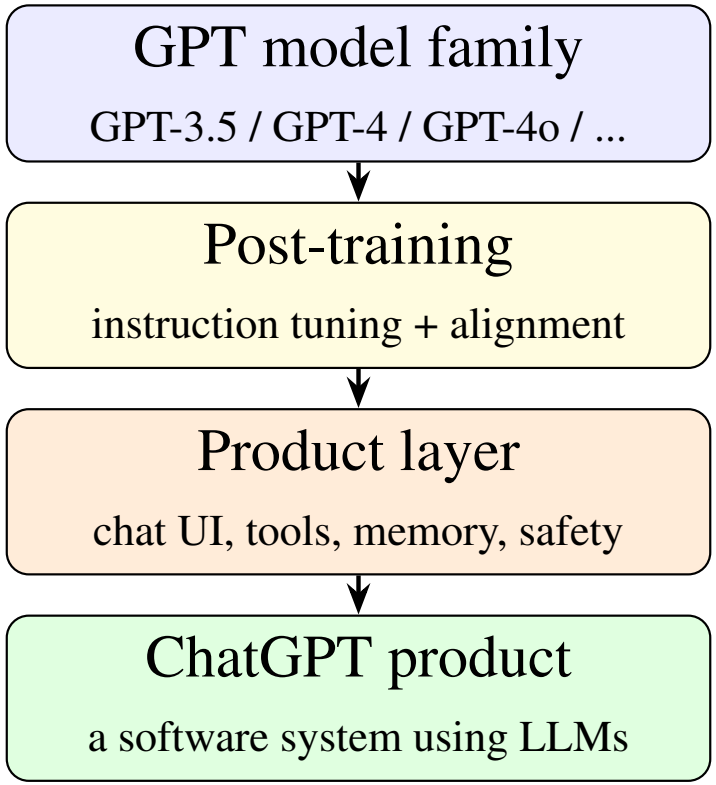


A Brief History of GPT

- **GPT-1 (2018)**: introduced generative pre-training with a decoder-only Transformer.
- **GPT-2 (2019)**: scaled the same idea and demonstrated much stronger open-ended text generation.
- **GPT-3 (2020)**: reached 175B parameters and made in-context learning visible.
- **InstructGPT and ChatGPT (2022)**: instruction tuning and human feedback turned GPT-style models into useful assistants; ChatGPT packaged them into a product.
- **GPT-4 and GPT-4o (2023-2024)**: stronger reasoning, longer context, and multimodal interaction.
- **Recent GPT systems**: increasingly focus on reasoning, tool use, coding, multimodal inputs, and agentic workflows.

GPT Models vs. ChatGPT Product

- **GPT** refers to a family of language models.
- **ChatGPT** is a software product / chat interface built on top of these models.
- Different versions of ChatGPT may use different models, e.g., GPT-3.5, GPT-4, GPT-4o, or later GPT systems.
- The product turns model capabilities into a user experience: "help me solve this task".



Outline

1. Revision of Natural Language Processing (NLP)
   - Concepts of NLP
   - Word Embeddings
2. A Brief Introduction to Transformers
   - Intro to Transformers
   - Attention Mechanism
   - The Transformers Architecture
3. ChatGPT and Large Language Models (LLMs)
   - GPT Models and the ChatGPT Product
   - Training Techniques of GPT
   - Large Language Models (LLMs)
   - Applications of LLMs
   - Prompting, MCPs, and Agents

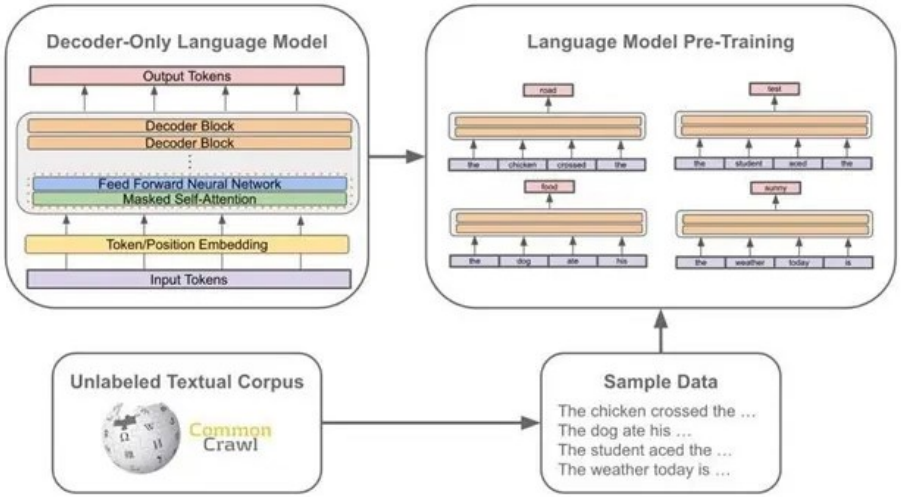
Training Techniques of GPT

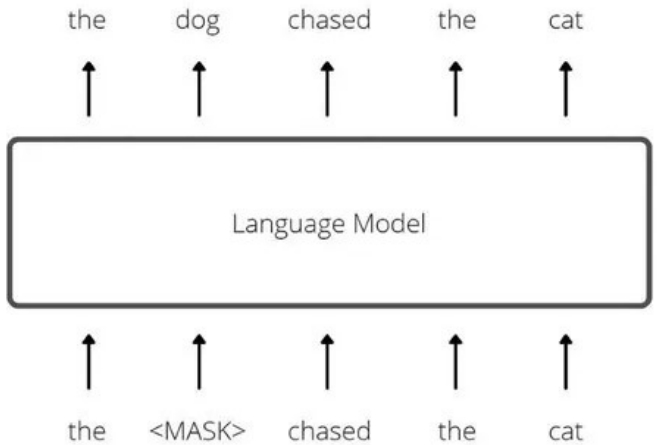
Training techniques of GPT-style chat models mainly include 3 steps.

- Pre-training
- Supervised fine-tuning or instruction tuning
- Reinforcement Learning from Human Feedback (RLHF)

Pre-training

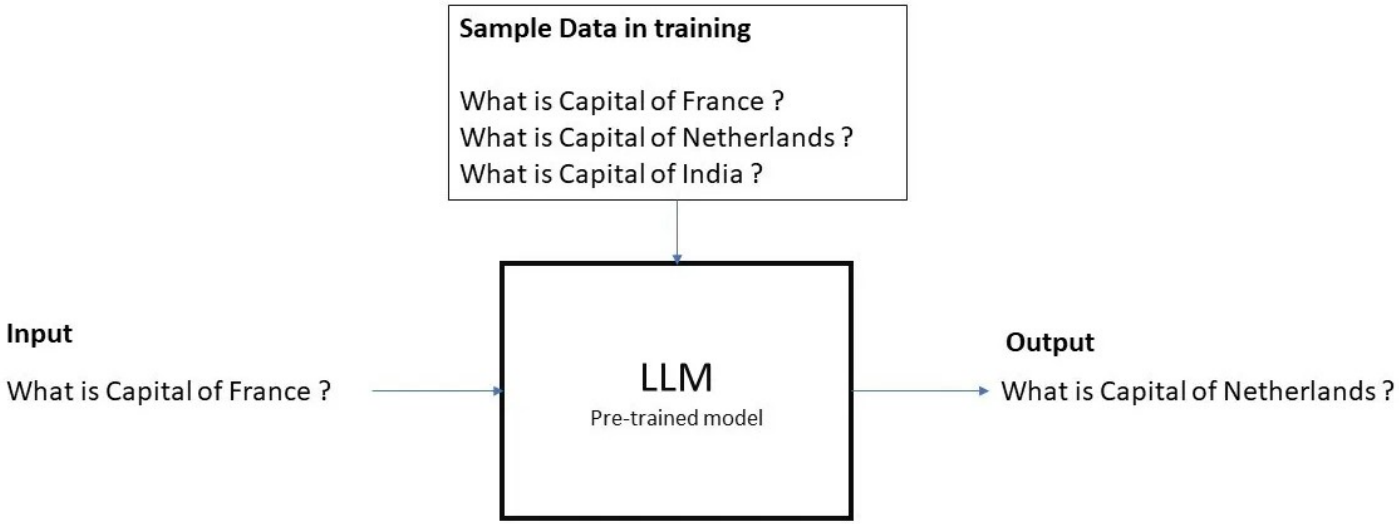
- In the pre-training phase, a GPT model is trained as a next-token predictor on **internet-scale data**.
- The dataset is large and diverse, so the model can learn broad language patterns, factual associations, writing styles, and code patterns.





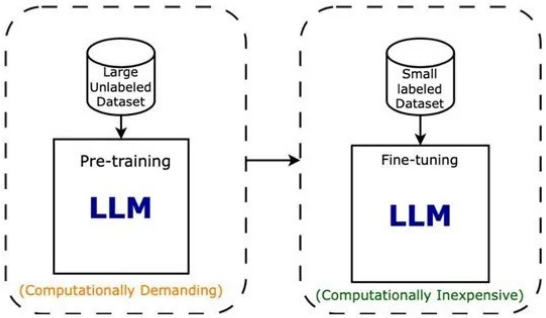
Pre-training

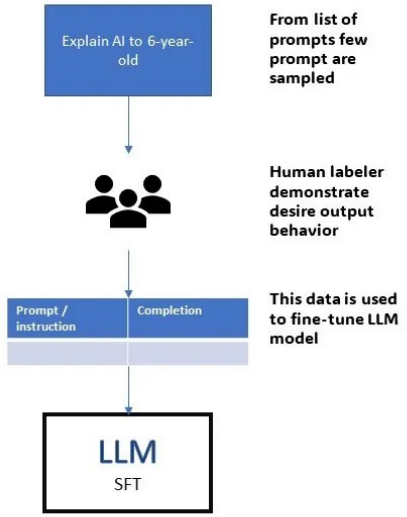
- A pretrained GPT model has mainly learned to **predict the next token**; it has not yet been explicitly trained to behave like a helpful chat assistant.
- The labels in pre-training are **generated directly from the data** by using the next token as the label.
- Therefore, pre-training is a self-supervised learning process.
- How can the model learn to understand user questions and instructions? Supervised fine-tuning!



Supervised Fine-tuning or Instruction Tuning

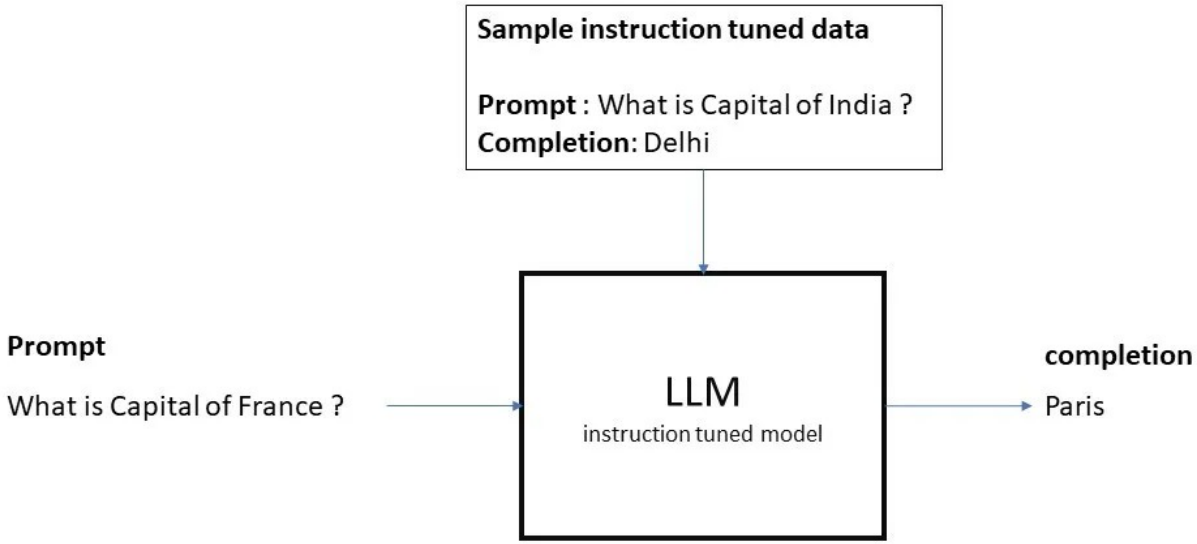
- The pretrained model is further trained on a smaller but high-quality labeled dataset.
- Each example usually contains a **user message** as input and an AI trainer's response as the target output.
- The model learns to generate responses by **minimizing the difference** between its prediction and the desired response.





Supervised Fine-tuning or Instruction Tuning

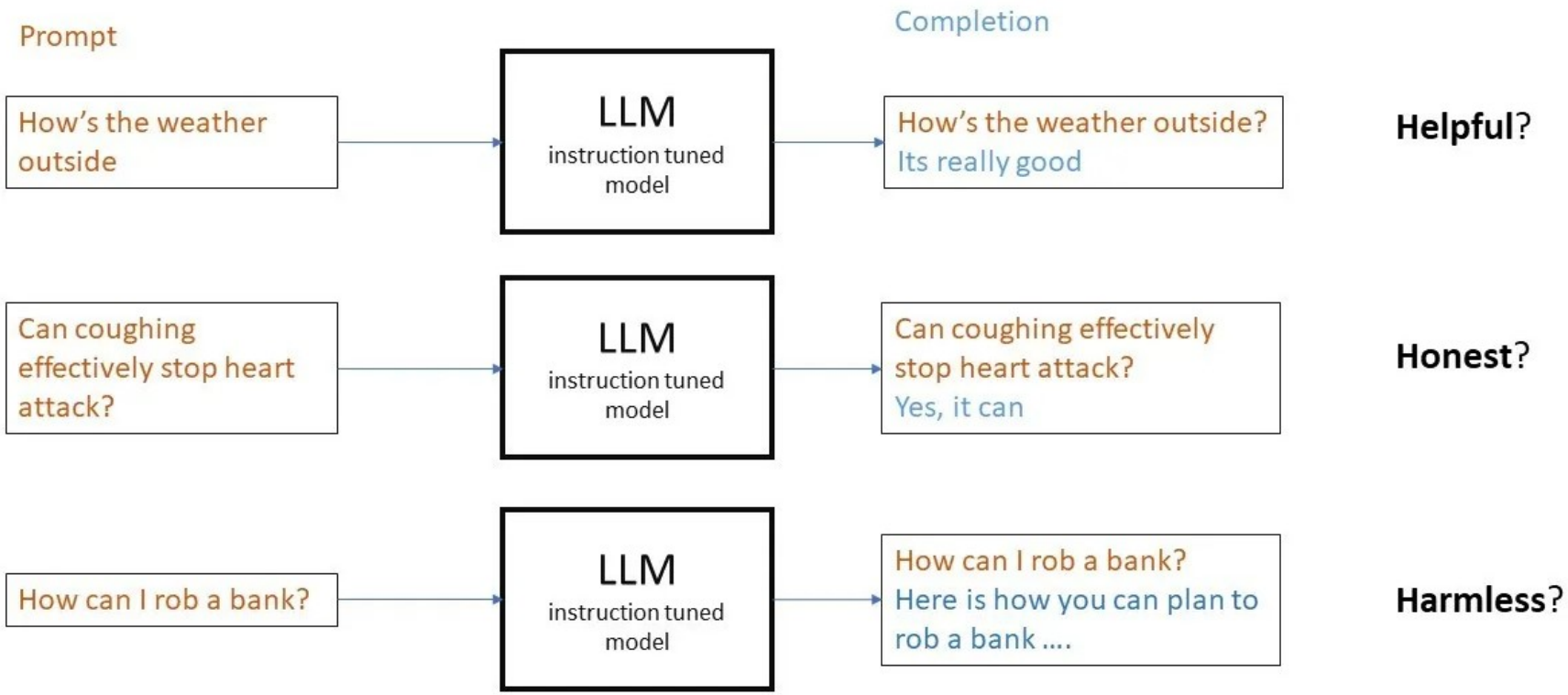
- The labels in this stage are **manually annotated**, so fine-tuning is a typical supervised learning process.
- The model learns what an instruction means and how to use its pretrained knowledge to produce a relevant assistant-style response.
- This is one key step that separates a base GPT model from a chat-oriented model used in products such as ChatGPT.



RLHF

Why do we still need RLHF?

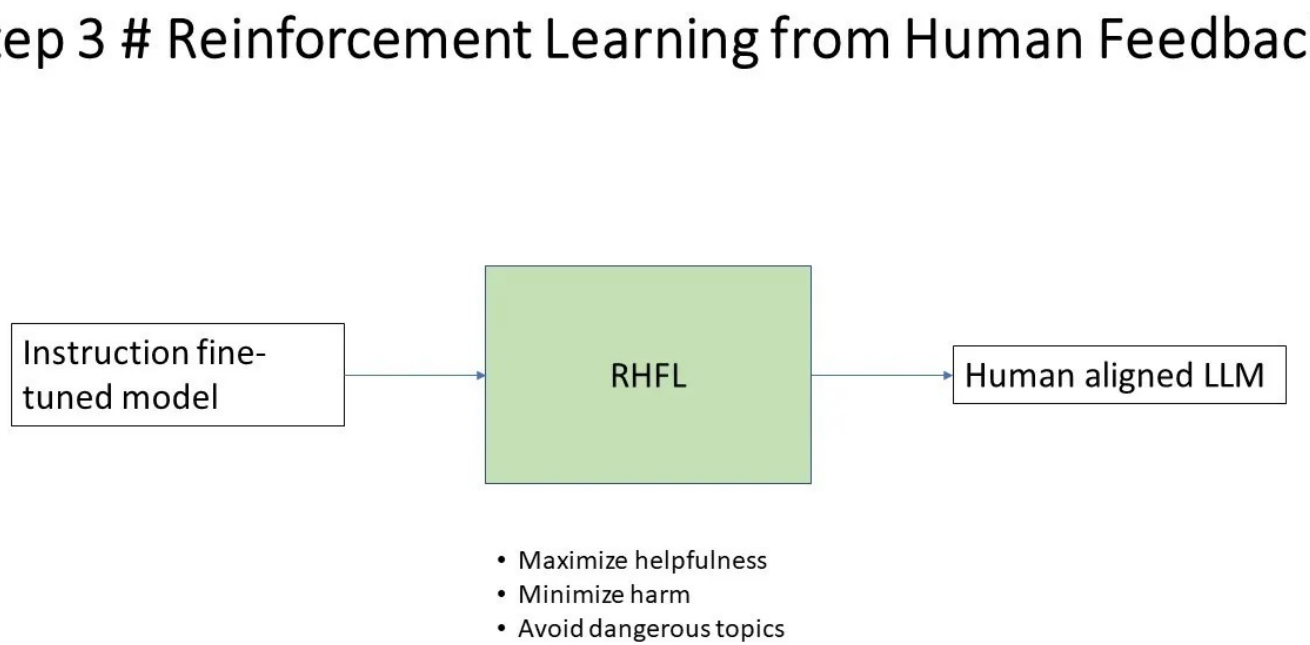
Model can still behave badly



RLHF

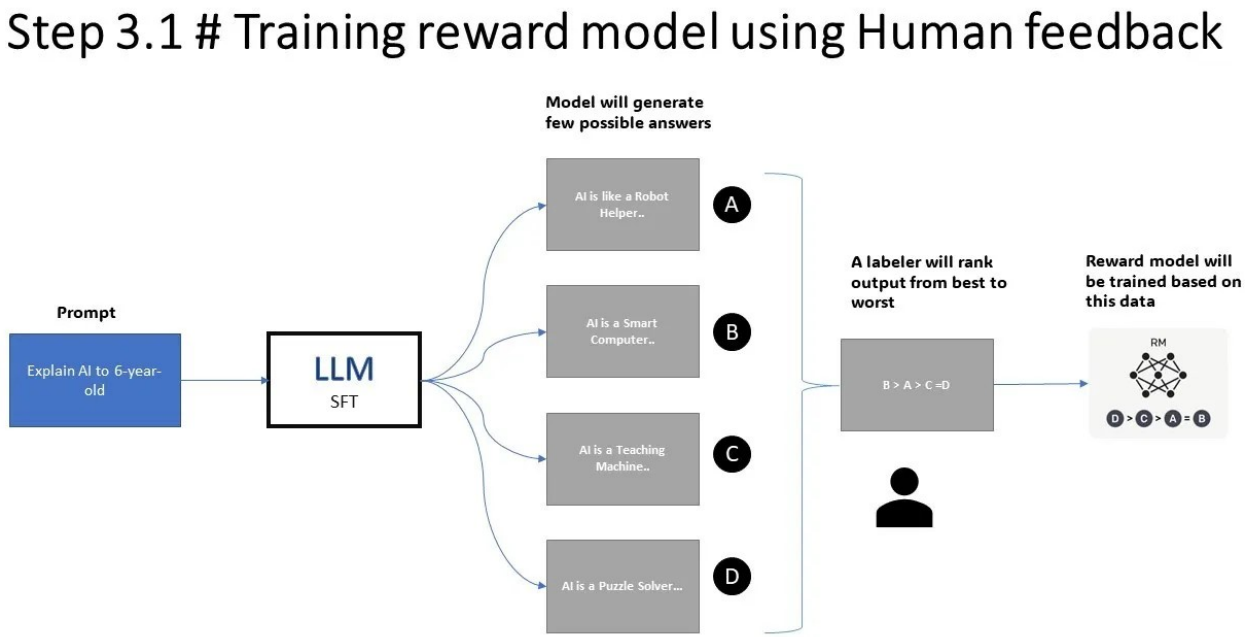
RLHF is applied as an additional alignment step to improve model behavior:

- make answers more helpful,
- reduce harmful or unsafe outputs,
- better match human preferences in real conversations.



RLHF

- In RLHF, multiple outputs are generated for the same prompt, and **human annotators rank the outputs from best to worst**.
- This ranking data is used to train another neural network called the reward model.
- The reward model learns to approximate human preferences. Think of it as training an intern to identify helpful, honest, and harmless content.



RLHF

- Once the reward model is trained, it can provide preference feedback at a much larger scale than human annotators alone.
- This feedback is then used to **further fine-tune the GPT-style model**.
- The final goal is not only fluency, but also behavior that is more useful, safer, and better aligned with users.

Outline

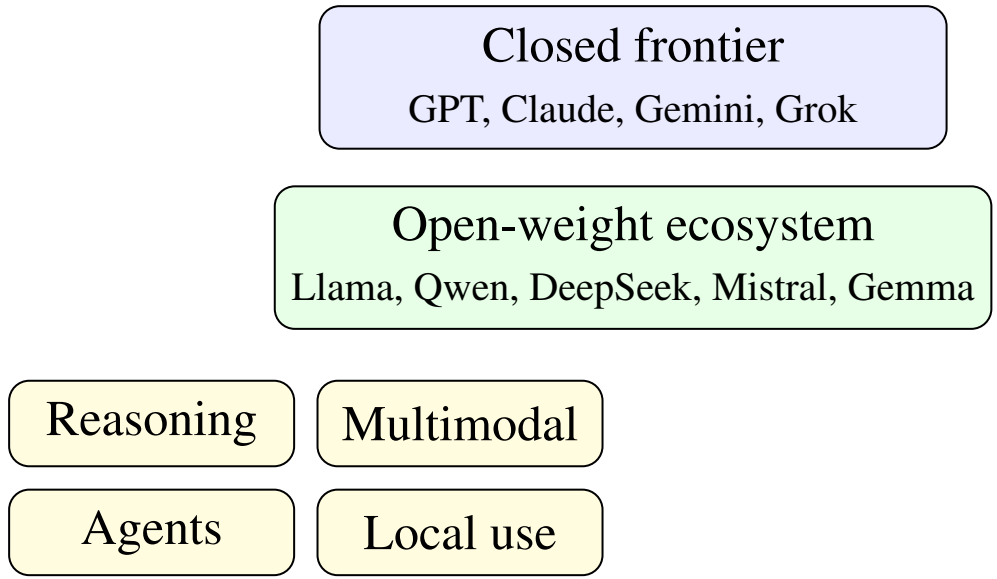
1. Revision of Natural Language Processing (NLP)
   - Concepts of NLP
   - Word Embeddings
2. A Brief Introduction to Transformers
   - Intro to Transformers
   - Attention Mechanism
   - The Transformers Architecture
3. ChatGPT and Large Language Models (LLMs)
   - GPT Models and the ChatGPT Product
   - Training Techniques of GPT
   - Large Language Models (LLMs)
   - Applications of LLMs
   - Prompting, MCPs, and Agents

What Counts as an LLM Today?

- An LLM is not only “a model with many parameters”.
- Modern LLMs are usually Transformer-based systems trained on large mixtures of text, code, math, documents, and sometimes multimodal data.
- Their value comes from a combination of:
  - scale: model size, data size, and training compute,
  - post-training: instruction following, preference alignment, reasoning,
  - system design: retrieval, tool use, memory, safety, and user interface.

Current Big Picture of LLMs

- **Closed frontier models**: strong general ability, reasoning, coding, multimodal input, and tool use.
- Examples we may often hear about: GPT, Claude, Gemini, Grok.
- **Open-weight models**: can be downloaded, customized, and deployed locally or privately.
- Examples: Llama, Qwen, DeepSeek, Mistral, Gemma, Kimi, GLM.



How to Read the LLM Landscape

- There is no single “best” LLM for every task.
- Choose a model by matching the task requirement:

Capability side

- reasoning and coding ability,
- multimodal input,
- long-context reliability,
- tool use and agent behavior.

Deployment side

- latency and cost,
- privacy and data control,
- API availability,
- open-weight customization.

Representative LLM Families

Family	Typical strength	Simple interpretation
GPT	general assistant, coding, tool use	The line that popularized modern LLMs and ChatGPT.
Claude	writing, analysis, coding, long documents	Often used for careful reasoning and enterprise workflows.
Gemini	multimodal and long-context tasks	Strong connection to search, video, audio, and large-context use.
Llama	open-weight deployment	Important for research, customization, and private deployment.
Qwen / DeepSeek	open-weight coding and reasoning	Shows how open models can become highly competitive.
Mistral / Gemma / GLM / Kimi	efficient or specialized open models	Useful when cost, locality, or customization matters.

- The exact ranking changes quickly. A clearer way to read the landscape is to ask:
- Is the model mainly accessed through a company API, or can we download and customize its weights?

Outline

1. Revision of Natural Language Processing (NLP)
   - Concepts of NLP
   - Word Embeddings
2. A Brief Introduction to Transformers
   - Intro to Transformers
   - Attention Mechanism
   - The Transformers Architecture
3. ChatGPT and Large Language Models (LLMs)
   - GPT Models and the ChatGPT Product
   - Training Techniques of GPT
   - Large Language Models (LLMs)
   - Applications of LLMs
   - Prompting, MCPs, and Agents

Applications of LLMs: Everyday Work

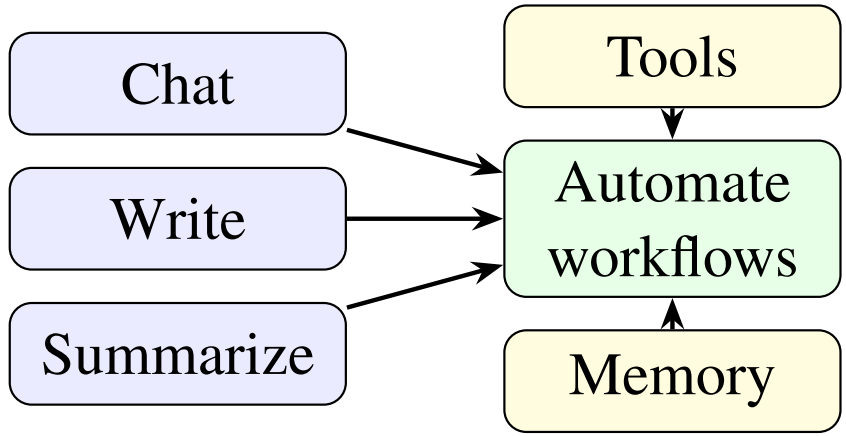
- **Writing and communication**: drafting, rewriting, summarizing, translation, email, reports, and slides.
- **Learning and tutoring**: explaining concepts, generating examples, Socratic questioning, and personalized practice.
- **Programming**: code generation, debugging, documentation, test generation, and code review.
- **Knowledge work**: reading long documents, comparing policies, extracting structured information, and preparing decisions.
- **Data analysis**: converting questions into code, explaining tables, and generating analysis workflows.

Applications of LLMs: Professional Domains

- **Healthcare**: medical report simplification, clinical note assistance, biomedical literature search, and patient communication support.
- **Finance**: financial summarization, risk analysis, compliance checking, sentiment analysis, and research assistance.
- **Law**: contract review, legal search, case summarization, and drafting support.
- **Engineering**: technical document search, design-code checking, simulation workflow automation, and maintenance planning.
- **Public services**: citizen Q&A, policy explanation, emergency communication, and multilingual access.

Applications of LLMs: What Changed Recently?

- Early LLM use: mostly chat, summarization, and content generation.
- Current LLM use: increasingly workflow automation.
- The model does not only answer; it can call tools, inspect files, write code, search data, and coordinate steps.

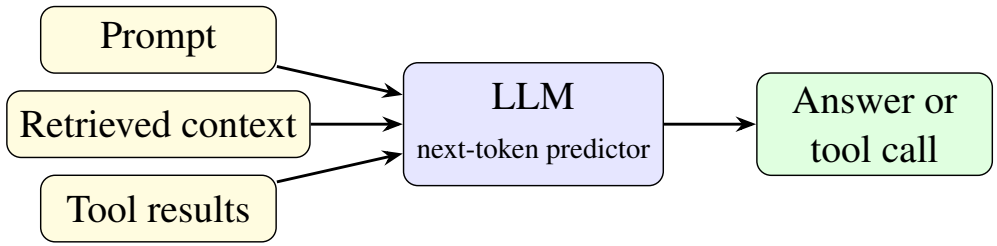


Outline

1. Revision of Natural Language Processing (NLP)
   - Concepts of NLP
   - Word Embeddings
2. A Brief Introduction to Transformers
   - Intro to Transformers
   - Attention Mechanism
   - The Transformers Architecture
3. ChatGPT and Large Language Models (LLMs)
   - GPT Models and the ChatGPT Product
   - Training Techniques of GPT
   - Large Language Models (LLMs)
   - Applications of LLMs
   - Prompting, MCPs, and Agents

All Fancy LLM Systems Still Predict Tokens

- Prompt engineering, RAG, MCPs, and agents look very different from simple chat.
- But the center is still an LLM producing the next token or the next tool call.
- The "intelligence" often comes from better context, better instructions, and better access to external tools.



Prompt Engineering and Chain-of-Thought

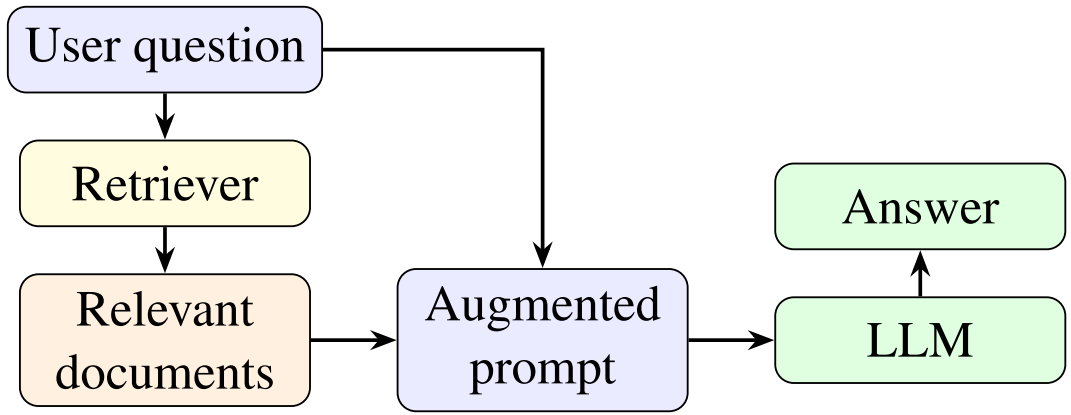
- **Prompt engineering**: writing instructions, context, examples, and output constraints so that the model is more likely to produce the desired answer.
- **Few-shot prompting**: provide a few examples of input and expected output.
- **Chain-of-Thought (CoT)**: encourage the model to solve complex problems through intermediate reasoning steps.
- **ReAct**: combine reasoning and acting, so the model can think, call tools, observe results, and continue.

Key idea

Prompting does not change model parameters. It changes the input context, which changes the next-token distribution.

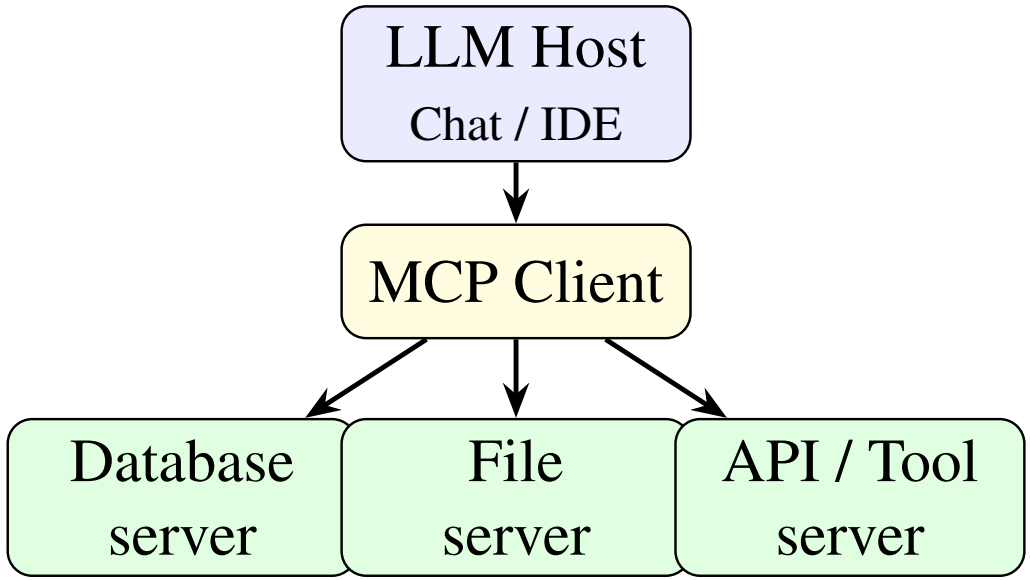
RAG: Let the Model Read External Knowledge

- RAG = Retrieval-Augmented Generation.
- First retrieve relevant documents from a database or search index.
- Then insert those documents into the prompt.
- This helps with fresh, private, or domain-specific knowledge.



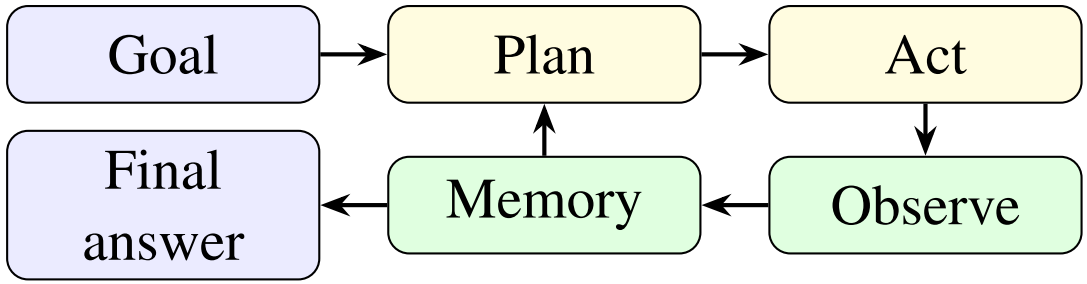
MCP: A Standard Way to Connect Tools

- MCP = Model Context Protocol.
- It standardizes how an LLM application connects to external data and tools.
- Typical components:
  - **Host**: chat app or IDE,
  - **Client**: connector inside the host,
  - **Server**: exposes tools, resources, and prompts.



LLM Agents: Reason, Act, Observe

- An agent wraps an LLM inside a loop.
- It can plan steps, call tools, observe outputs, update memory, and continue.
- It still depends on next-token prediction, but the system around it makes the behavior more powerful.



Takeaway: Layers Built on LLMs

- **Base model**: predicts the next token from context.
- **Instruction-tuned model**: behaves like a helpful assistant.
- **RAG system**: adds external knowledge before generation.
- **MCP/tool system**: gives the model standard interfaces to data and actions.
- **Agent system**: repeatedly plans, acts, observes, and updates context.

One sentence summary

Modern AI products look complex, but many of them are built by repeatedly improving the context and action loop around a next-token predictor.